

\$ KUBECTL APPLY

Kubernetes

101

El arranque del libro

Antes de empezar · Arquitectura de Kubernetes
Apéndice D: El clúster por dentro

Veintiséis páginas y los ocho diagramas, tal y como aparecen en el libro.

REGALO PARA SUSCRITORES

Javier Vela Aylón

www.javivela.dev · [@jvela](https://twitter.com/jvela)

Antes de empezar

Tienes una aplicación web. Se ejecuta en un servidor. Funciona.

Un martes a las tres de la mañana el proceso muere. Nadie se entera hasta las nueve, cuando un cliente llama. Entrás por SSH, lo arrancas otra vez y te vas a almorzar pensando que habría que montar algo que lo levante solo.

Montas ese algo: un script que cada minuto mira si el proceso está vivo y, si no lo está, lo arranca. Ha dejado de ser tu problema a las tres de la mañana. Acabas de escribir, sin saberlo, la idea central de Kubernetes: tú declaras lo que quieres que haya, y algo se encarga de que lo haya. No das pasos, das un resultado. Lo que decide qué hacer para llegar hasta ahí es el sistema, y no para nunca de comprobarlo.

Pero un servidor sigue siendo un servidor. El día que se cae la máquina entera, tu script se cae con ella. Necesitas la aplicación en tres máquinas. Y en cuanto tienes tres copias aparece una pregunta nueva: ¿a cuál llaman los demás? No puedes repartir tres direcciones IP por ahí, porque esas copias van a morir y a nacer constantemente —esa es justo la gracia— y ninguna dirección va a durar. Hace falta un nombre fijo que apunte siempre a las que puedan atender las peticiones.

Y hay una tercera cosa. Lo de las copias intercambiables está muy bien para la web, pero tu base de datos no es intercambiable: tiene un disco que no puede perder y un nombre que no puede cambiar. Da igual cuántas copias levantes, no son la misma cosa que las anteriores.

Kubernetes es ese script, hecho bien, para un montón de máquinas a la vez. Y todo él se apoya en tres ideas:

- **Declaras el resultado, no los pasos.** Escribes que quieres tres copias de algo y quién puede hablar con quién. Kubernetes compara constantemente lo que hay con lo que pediste, y si no coincide, actúa. No hay un botón de “desplegar”: hay un estado deseado y algo que lo persigue.
- **Lo que se ejecuta es un Pod, y un Pod es de usar y tirar.** Es la unidad más pequeña que Kubernetes sabe arrancar: uno o varios contenedores que comparten red y disco, y que nacen y mueren juntos. Un Pod no se repara, se sustituye; que muera no es una avería, es el funcionamiento normal. Y como

se sustituye, hacen falta dos piezas más: alguien que reponga los que falten y un nombre fijo al que llamar, porque las direcciones de los Pods no duran.

- **Pero no todo es sustituible.** Tu base de datos tiene un disco que no puede perder y un nombre que no puede cambiar: dos copias tuyas no son lo mismo que dos copias de la web. Saber qué pieza es de usar y tirar y cuál no está detrás de la mitad de las decisiones de este libro.

Con esas tres ideas ya puedes leer el mapa. Los nombres que vas a ver a partir de aquí —Deployment, StatefulSet, Service, DaemonSet— son los objetos con los que Kubernetes construye todo esto. Cada uno tiene su capítulo; de momento basta con saber a qué pieza pertenece cada nombre.

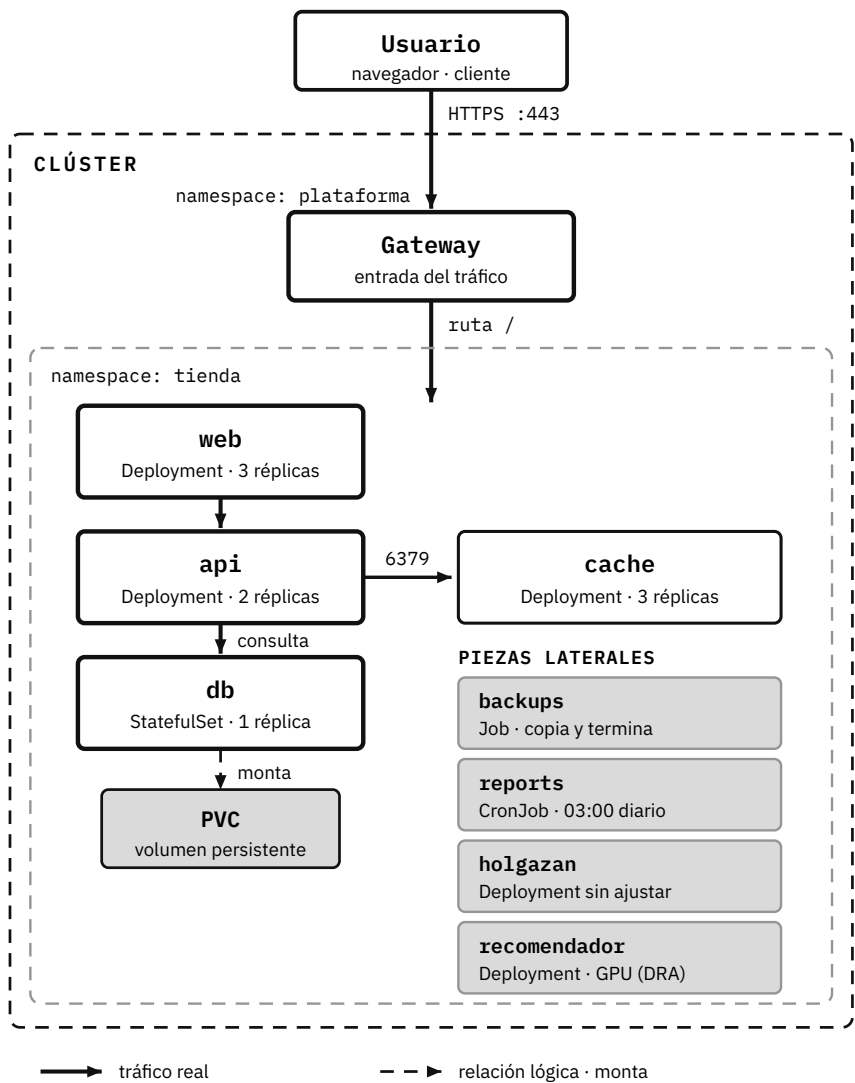


Figura 1: La aplicación tienda: Namespaces y piezas

Sigue el trazo grueso y tendrás el camino de una petición: Usuario, Gateway, web, api y db. Falta una pieza en este mapa, el agent: es la única que no vive en el Namespace tienda, y por eso aparece en la figura siguiente.

Las piezas

Son seis, y cada una está aquí porque enseña algo que las demás no pueden enseñar. Antes de mirarlas una a una, quédate con la división que las ordena a todas: hay piezas que puedes tirar sin que pase nada, y hay una que no.

web — *la que se puede tirar sin consecuencias*. Es el frontend, tres copias idénticas. A cualquiera de ellas le da exactamente igual desaparecer, y esa indiferencia es justo lo que permite cambiarle la versión sin cortar el servicio. Por eso es un Deployment.

api — *la que crece sola*. Es el backend y la pieza que sufre los picos de tráfico. Todas las demás tienen el tamaño que tú les pusiste; esta se lo pone ella, según la gente que llegue.

db — *la que no puede perder su disco*. Es PostgreSQL: un disco que no puede perder, un nombre que no puede cambiar y una sola copia que no es sustituible por ninguna otra. Por eso no es un Deployment, es un StatefulSet. Si entiendes por qué esta pieza es distinta de las dos anteriores, entiendes la mitad de Kubernetes.

cache — *la que obliga a decidir en qué nodo cae cada copia*. Es Redis, tres copias, y hay tres nodos. La pregunta parece tonta hasta que la haces: ¿quién garantiza que caiga una en cada uno? Nadie, hasta que lo declaras.

agent — *la que va en todas las máquinas*. Recoge los logs, y hay una instancia por nodo en vez de una por aplicación. Necesita leer el disco del nodo, y esa necesidad la destierra: es la única pieza que no vive en el Namespace `tienda`.

gateway — *la que deja pasar el tráfico de fuera*. Es la puerta de entrada, y donde termina el TLS¹. Vive en un Namespace aparte a propósito, porque quien abre la puerta no debería estar dentro de la casa.

A esas seis se les suman cuatro que no están en el mapa: aparecen cuando un capítulo las necesita.

- **backups** es un Job: copia la db y termina. Empezar y terminar es justo lo que distingue una tarea de un servicio.
- **reports** es el informe de ventas, y es un CronJob: la misma idea, pero cada noche a las tres y sin que nadie se lo pida.

¹Transport Layer Security. Protocolo criptográfico que cifra los datos intercambiados entre dos aplicaciones a través de una red, como Internet.

- **holgazán** pide dos CPU y cuatro gigas de memoria para usar una décima parte. Existe para que el VPA² tenga algo que arreglar, y para que veas cuánto se despilfarra cuando nadie mira los números.
- **recomendador** necesita una GPU. Es el único que pide hardware en vez de solo CPU y memoria, y en el clúster solo hay una: por eso es con lo que se explica DRA³.

El clúster tiene tres nodos y tres Namespaces: tienda para la aplicación, plataforma para lo que deja entrar tráfico o toca el nodo, y observabilidad para Prometheus, Grafana y Loki. Los nombres exactos de cada objeto, sus imágenes y sus puertos están en el Apéndice B, junto con el sistema entero en YAML, en el repositorio del libro.

²VerticalPodAutoscaler. Ajusta automáticamente las `requests` y `limits` de CPU y memoria de un Pod según su consumo real.

³Dynamic Resource Allocation. Mecanismo para solicitar dispositivos especializados (GPUs, FPGAs, NICs...) con requisitos concretos (modelo, memoria, topología). El usuario describe lo que necesita en un `ResourceClaim` y el `kube-scheduler` elige nodo y dispositivo a la vez.

LOS 3 NODOS DEL CLÚSTER

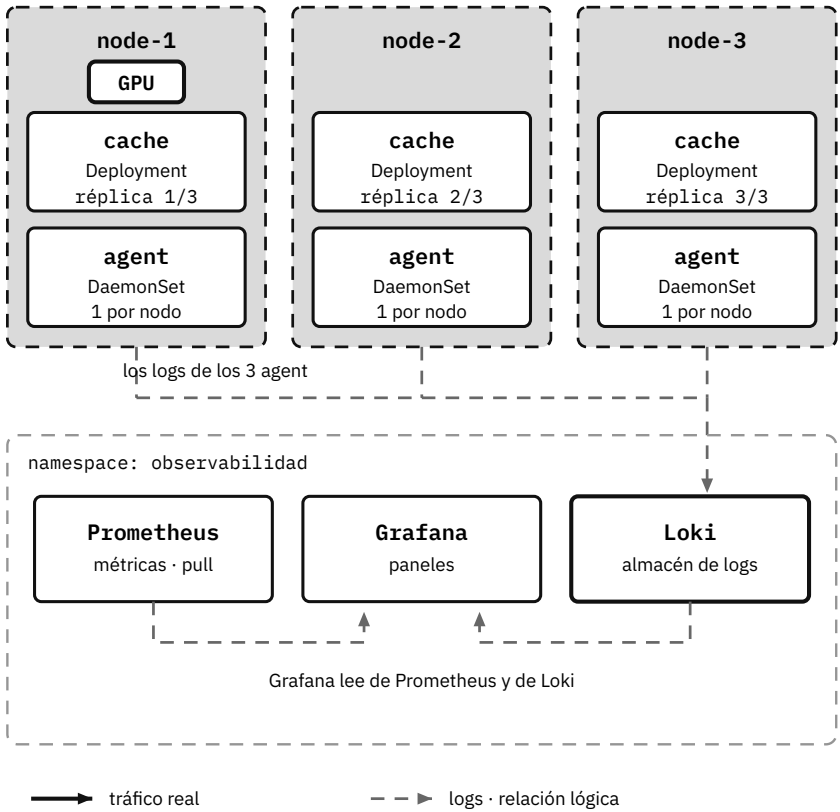


Figura 2: La aplicación tienda: los tres nodos y la observabilidad

Solo node-1 tiene GPU. Esa escasez es la razón de ser de DRA: recomendador la pide con un ResourceClaim y, si otro Pod la quisiera a la vez, uno de los dos esperaría.

Cómo se construye

Este es el recorrido del sistema, capítulo a capítulo. No es el índice completo – los primeros capítulos son de conceptos y no añaden piezas–, pero sí está todo lo que construye la tienda. Si en algún momento te pierdes, vuelve aquí:

Capítulo	Qué añade al sistema
Objetos de Kubernetes	El Namespace tienda y web, todavía como un Pod suelto
Workloads	web pasa a Deployment · llega db (StatefulSet) · agent (DaemonSet) · backups y reports
Escalado	api aprende a escalar sola (HPA) · aparece el holgazán (VPA)
Red	Los Services de web, api y cache · el gateway · la NetworkPolicy que aísla la db
Configuración	web-config y el Secret db-credentials
Almacenamiento	El disco de la db: pv-db y datos-db-0
Seguridad	agent-sa, RBAC y el endurecimiento de api
Políticas	El presupuesto del Namespace tienda
Scheduling	El Deployment de cache, una réplica por nodo · el PDB de web · el recomendador y su GPU
Trabajando con el clúster	Operar todo lo anterior
Extensibilidad	Ninguna pieza nueva: el tipo Promoción y el operador que lo atiende
Observabilidad	El Namespace observabilidad

El sistema entero, en YAML y listo para aplicar, está en el repositorio del libro: <https://github.com/fjvela/kubernetes-101>. El Apéndice B te dice qué hay allí y cómo desplegarlo.

Versión de referencia

Todo lo descrito en este libro corresponde a Kubernetes 1.37 (agosto de 2026), salvo que se indique lo contrario. Kubernetes evoluciona rápido: algunas funciones que aquí aparecen como estables pueden haber cambiado, y otras que se mencionan

como novedad pueden haberse consolidado. Cuando una versión concreta importa, se indica en el texto.

Erratas y actualizaciones. Las correcciones posteriores a la impresión y los cambios de versión que afecten a lo que aquí se cuenta se publican en <https://www.javivela.dev/libros/kubernetes-101/erratas>

Parte práctica. Casi todos los capítulos tienen su laboratorio en un clúster de Kubernetes real que arranca **en tu navegador**: no hay nada que instalar, ni Docker, ni un clúster que montar. Empieza por el módulo *Fundamentos*, donde creas el primer Pod de la tienda, lo rompes y lo diagnosticas. Al final de cada capítulo encontrarás a qué módulo ir.

Las condiciones están en la última página del libro.

Arquitectura de Kubernetes

La arquitectura de Kubernetes permite orquestar y ejecutar contenedores a escala de una manera distribuida.

Para ello emplea un conjunto de componentes que interactúan entre sí a través de una API para mantener el estado definido por los usuarios.

Estos componentes se agrupan en dos grandes bloques:

- El control plane, que toma las decisiones globales sobre el clúster: qué Pods se ejecutan, dónde se ejecutan y cómo reaccionar cuando algo cambia respecto al estado deseado.
- Los nodos, las máquinas (físicas o virtuales) donde realmente se ejecutan los Pods.

En clústeres desplegados con herramientas como `kubeadm`²³, los propios componentes del control plane se ejecutan como Pods sobre los nodos de control plane. Por eso, estos nodos también necesitan ejecutar `kubelet`²⁴, aunque no alojen cargas de trabajo de usuario.

De todos ellos, dos hay que tener en la cabeza desde la primera página, porque el resto del libro los da por sabidos: el `kube-apiserver`, por donde entra absolutamente todo, y el `kubelet`, que es quien acaba ejecutando. Son los dos que vienen a continuación.

Los demás se explican donde hacen falta. `kube-proxy` abre el capítulo de Red, porque no se entiende sin el Service. El `kube-scheduler` abre el de Scheduling, que es donde se decide dónde cae cada Pod. Y la admisión vive en el de Seguridad, pegada a las políticas que la usan. Lo que queda —`etcd`, el inventario de controladores, la interfaz con el runtime de contenedores y los Leases— está en el Apéndice D, que puedes leer hoy o el día que lo necesites.

²³`kubeadm`. Herramienta oficial para instalar y arrancar un clúster de Kubernetes conforme a las buenas prácticas. Despliega los componentes del control plane como Pods sobre los nodos de control.

²⁴`kubelet`. Agente que se ejecuta en cada nodo. Recibe del API Server los PodSpec que le corresponden, pide al runtime que arranque los contenedores y reporta el estado del nodo y de los Pods.

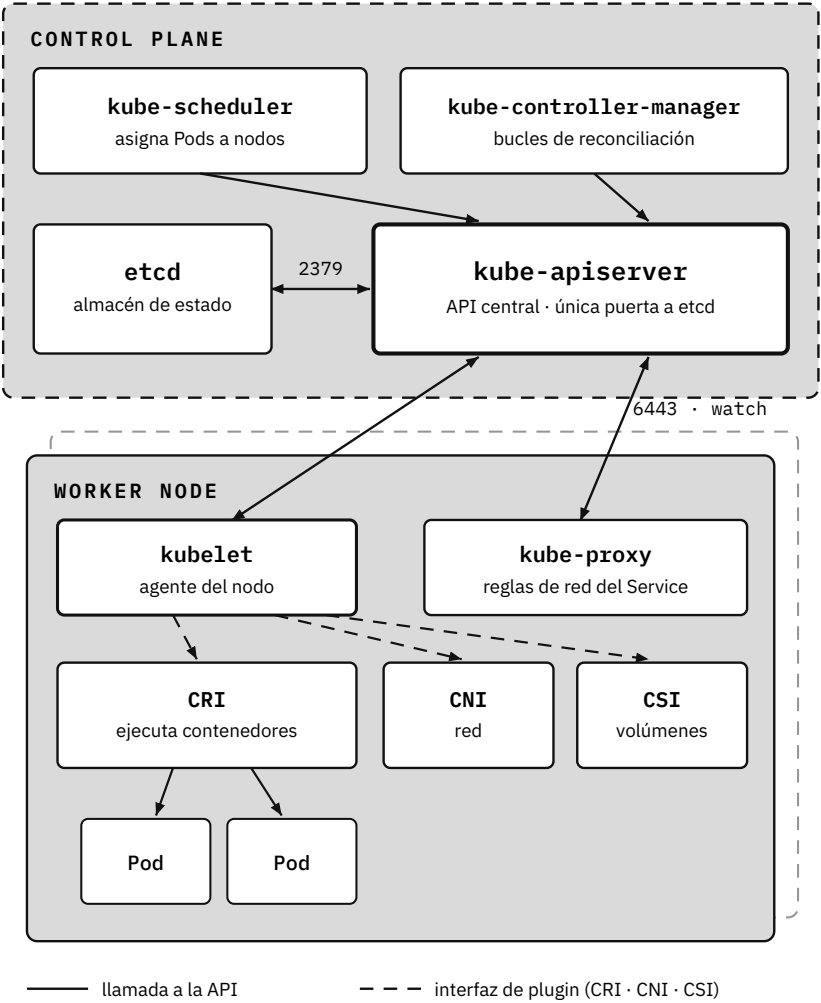


Figura 3: Arquitectura del clúster: control plane y nodos

Fíjate en que todas las flechas pasan por el kube-apiserver: ningún componente habla directamente con otro, ni siquiera los que viven en la misma máquina. Eso es lo que hace que solo haya un punto que proteger de verdad, y también que solo haya uno que se pueda saturar.

Componentes clave

kube-apiserver

El kube-apiserver es el punto por el que entran todas las peticiones del clúster, tanto las del resto de componentes como las de sus usuarios, ya sea para crear objetos nuevos o para consultar su estado. Se interactúa con él mediante una API REST, y es también quien almacena y actualiza esa información en etcd.

Antes de ejecutar una petición, esta es autenticada, autorizada y analizada por los admission controllers. En las siguientes páginas describiremos cada uno de estos elementos.

Por defecto, el puerto utilizado por la API es el 6443 y está protegido por TLS¹.

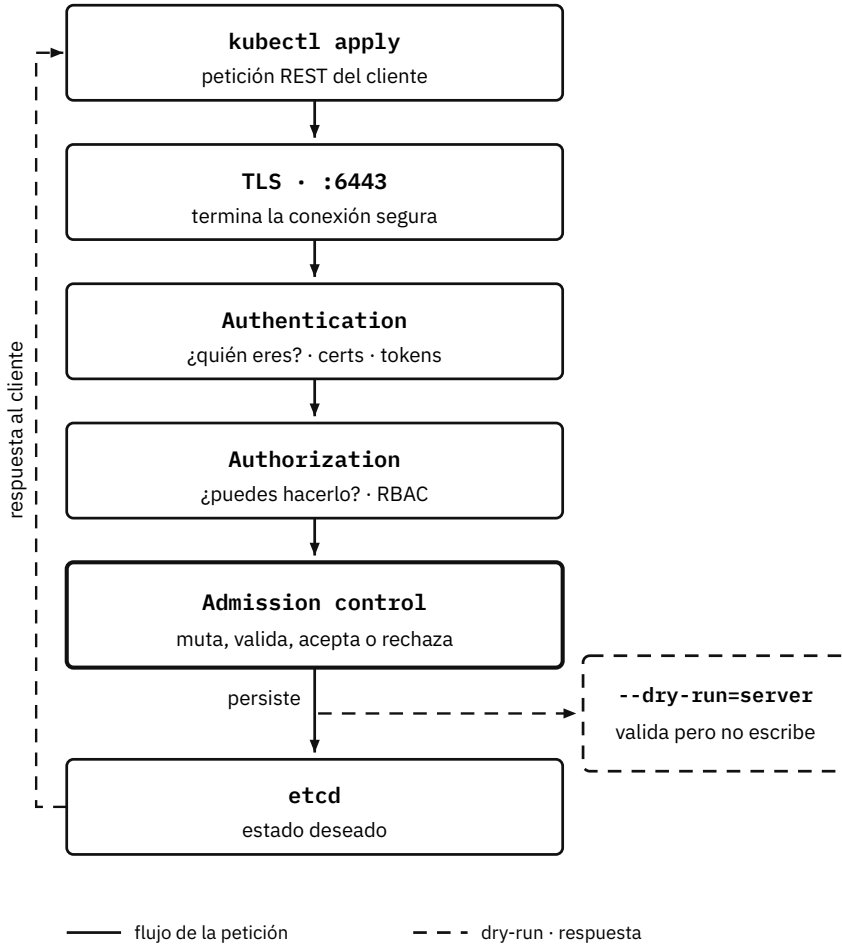


Figura 4: El ciclo de una petición en el kube-apiserver

Saber qué etapa te ha rechazado ahorra medio diagnóstico: un Unauthorized viene de la autenticación, un Forbidden de la autorización y casi cualquier otro mensaje raro, de la admisión. `--dry-run` recorre todo el pipeline, admisión incluida, pero no se guarda en etcd.

Es posible configurar más de un método de autenticación. Al menos deben existir dos métodos:

- Uno para autenticar las peticiones que provienen de las Service Accounts (SA)
- Otro para autenticar a los usuarios del clúster

En el caso de que haya más de un módulo de autenticación activado, se van ejecutando secuencialmente hasta que uno de ellos puede validar correctamente la petición.

Los módulos de autenticación y autorización se explican en el capítulo de Seguridad.

kubelet

kubelet se ejecuta como una aplicación en cada uno de los nodos que componen el clúster, incluidos los nodos de control plane en clústeres autogestionados.

Sus responsabilidades principales son:

- Registrar el nodo en el control plane.
- Obtener las especificaciones de los Pods que debe ejecutar.
- Crear, modificar y borrar los contenedores asociados a un Pod, delegando la ejecución real en el container runtime a través de la Container Runtime Interface (CRI).
- Manejar las diferentes probes (liveness, readiness y startup) configuradas para cada uno de los contenedores.
- Informar al control plane sobre el estado de los Pods que se están ejecutando en el nodo.

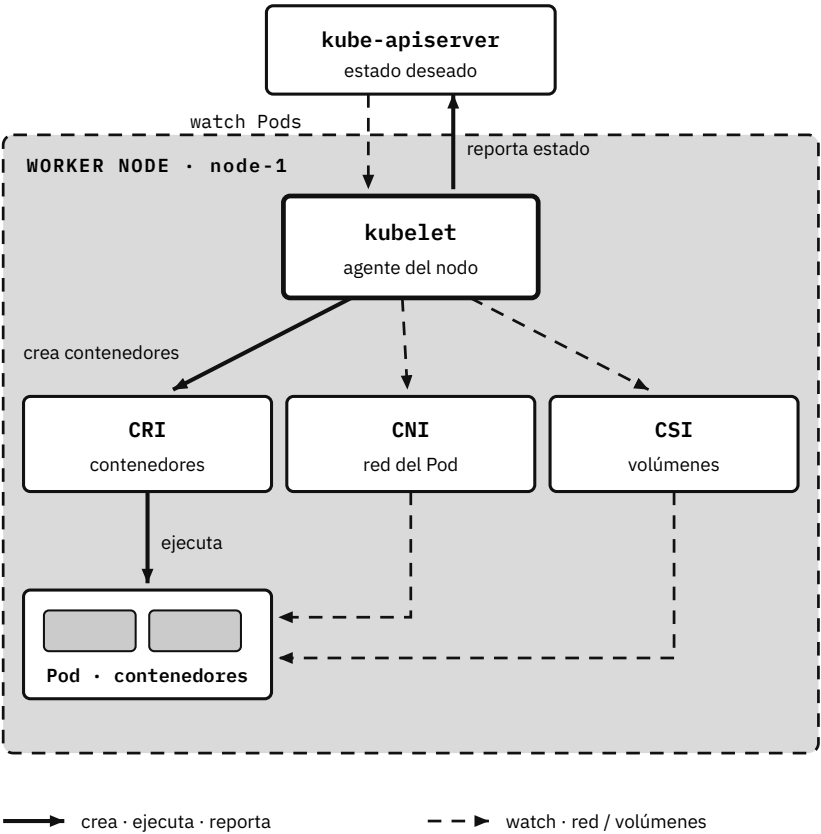


Figura 5: El kubelet y los Pods de su nodo

El kubelet no espera órdenes: vigila el API server y actúa sobre lo que le toca. Por eso un nodo que pierde la conexión con el control plane sigue ejecutando sus Pods tan tranquilo; lo que deja de haber no son contenedores, son cambios.

Comunicación entre componentes

Kubernetes implementa el patrón hub-and-spoke²⁵, en el que el kube-apiserver actúa como único punto central de comunicación. Salvo el propio API Server y la excepción del kubelet (que verás a continuación), ningún componente expone puertos para recibir conexiones remotas de forma directa.

Acceso al API Server

Se accede principalmente mediante HTTPS. Desde dentro del clúster se hace a través del Service kubernetes, que escucha en el puerto 443 y redirige al 6443 del API server; desde fuera, directamente al 6443. Soporta múltiples estrategias de autenticación: certificados X.509, tokens de ServiceAccount, OpenID Connect, webhook, etc.

Autenticación de nodos y Pods

Los nodos se autentican con certificados de cliente (generados normalmente por kubeadm o herramientas similares).

Los Pods utilizan ServiceAccounts. Kubernetes monta automáticamente el token y el certificado CA en cada Pod, permitiéndole comunicarse de forma segura con el API Server.

Comunicación entre el API Server y kubelet

Esta comunicación es bidireccional y se utiliza para:

- Obtener logs de los Pods (`kubectl logs`).
- `kubectl port-forward`.
- `kubectl attach/kubectl exec`.
- Recopilación de métricas y estado del nodo.

Se establece sobre el puerto HTTPS del kubelet (10250), el único puerto que un componente distinto del API Server expone para recibir conexiones remotas directas. Por defecto, el API Server no valida el certificado del kubelet (paráme-

²⁵Hub-and-spoke: Topología de comunicación en la que todas las piezas hablan con un único punto central (el hub) y nunca entre sí (los spokes). Reduce el número de conexiones que hay que asegurar y deja un solo sitio donde autenticar, autorizar y auditar; a cambio, ese centro es crítico y conviene desplegarlo en alta disponibilidad. En Kubernetes el hub es el kube-apiserver.

tro `--kubelet-certificate-authority` vacío), lo que representa un riesgo de seguridad que debe mitigarse en producción.

Otras conexiones del API Server

El API Server también puede iniciar conexiones hacia:

- Nodos (a través del kubelet).
- Pods (usando el kubelet como proxy).
- Services (para proxying interno).

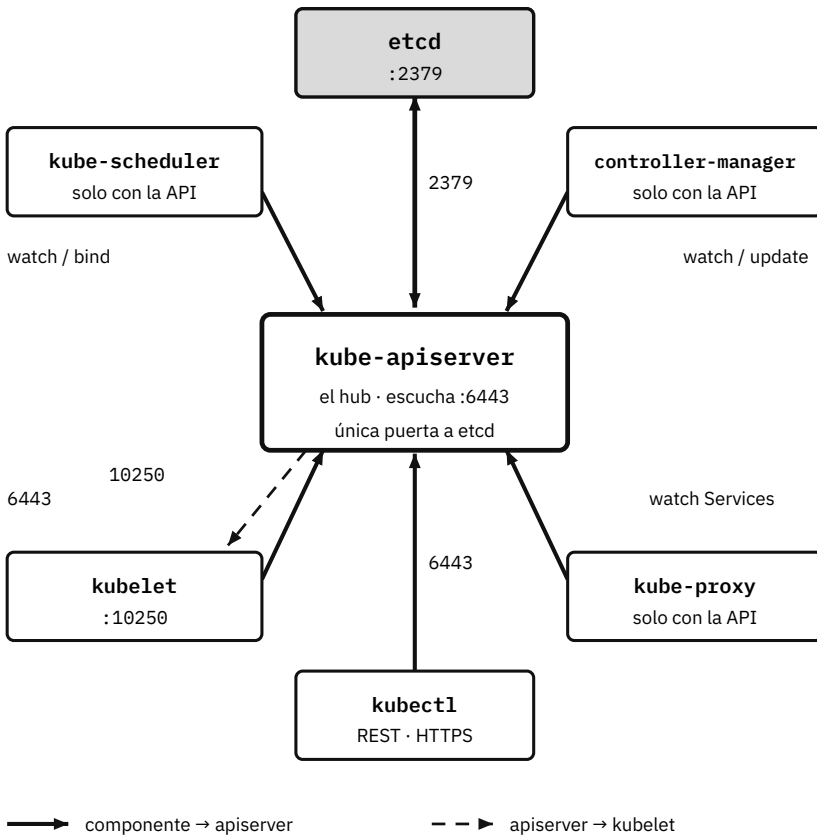


Figura 6: Comunicación entre los componentes del clúster

Ningún componente habla con otro directamente y solo el API Server toca etcd. Por eso un cortafuegos entre nodos apenas necesita abrir dos puertos: 6443 hacia el control plane y 10250 hacia los kubelets.

Laboratorio. Módulo *El clúster por dentro*: un recorrido por un clúster kubeadm componente a componente, con lo que aquí se cuenta y lo que amplía el Apéndice D.

Apéndice D: El clúster por dentro

El capítulo de Arquitectura se queda con lo que hay que tener en la cabeza para leer el resto del libro: quién recibe las peticiones y quién ejecuta los contenedores. Aquí está lo que hay por debajo.

No hace falta para desplegar una aplicación, y por eso no está en el cuerpo del libro. Hace falta el día que un clúster va lento y nadie sabe por qué, cuando toca dimensionar el control plane, o cuando hay que actualizar de versión y conviene saber qué se mueve por dentro.

etcd

En todo momento Kubernetes monitoriza, analiza y realiza las operaciones necesarias para poder mantener el estado deseado por los usuarios.

Para almacenar esta información hace uso del almacén de clave-valor distribuido `etcd`.

Al estar distribuido en varias instancias, `etcd` ofrece alta disponibilidad: si una instancia falla, el resto puede seguir respondiendo peticiones sin interrupción. Esto es distinto de recuperarse de un fallo catastrófico de todo el clúster, que depende de disponer de backups de `etcd` (lo vemos un poco más abajo).

Las principales características de `etcd`:

- **Consistencia:** utiliza el algoritmo Raft³², garantizando que los nodos tengan la misma versión de los datos en todo momento.
- **Alta disponibilidad:** los datos se pueden replicar en múltiples instancias, lo que permite seguir accediendo a los datos en caso de que una instancia falle.
- **Interfaz simple:** desde la versión 3, `etcd` expone su API de forma nativa mediante gRPC³³, con operaciones CRUD (create, read, update, delete). También

³²Raft. Algoritmo de consenso distribuido que utiliza `etcd` para elegir un líder y garantizar que todas las réplicas mantienen los mismos datos. Explicación interactiva: <https://raft.github.io/>

³³gRPC. Protocolo de llamada a procedimiento remoto (RPC) sobre HTTP/2 que serializa los mensajes en formato binario (Protocol Buffers). Es más eficiente que REST/JSON y admite streaming bidireccional.

existe un gateway que permite acceder a esta misma API a través de HTTP/JSON.

- **Watch API:** permite suscribirse a cambios en claves específicas para obtener notificaciones cuando se produce un cambio en los valores de la clave.

`etcd` es una parte crítica de Kubernetes debido a algunas de sus limitaciones que conviene tener en cuenta:

- **Latencia:** una alta latencia puede provocar timeouts y cambios frecuentes en el líder (leader).
- **Acceso a disco:** es recomendable la utilización de discos SSD. El rendimiento de `etcd` está muy relacionado con el rendimiento del disco utilizado.
- **Limitaciones de tamaño:** el tamaño aproximado de un único valor (value) es de ~1 MiB. Por defecto, el tamaño máximo del almacén es de 2 GB, ampliable hasta un máximo recomendado de 8 GB. El tiempo de arranque es proporcional al tamaño de los datos. A mayor tamaño, mayor tiempo de arranque.

Para finalizar, y en caso de que no utilices un clúster de un cloud provider, debes aplicar las siguientes prácticas:

Backups: realizar backups frecuentes permite restaurar el estado del clúster en caso de tener un fallo catastrófico. `etcdctl` puede ayudarte en esta tarea.

Monitorización: tener configurada una correcta monitorización del uso de CPU, memoria y espacio en disco te ayuda a detectar de manera proactiva un posible problema.

Alta disponibilidad: utilizar varias instancias para ejecutar el servicio evita tener un punto único de fallo.

`etcd` es un proyecto que forma parte de la Cloud Native Computing Foundation (CNCF - <https://www.cncf.io/>), puedes leer más sobre él en su página web <https://etcd.io/>.

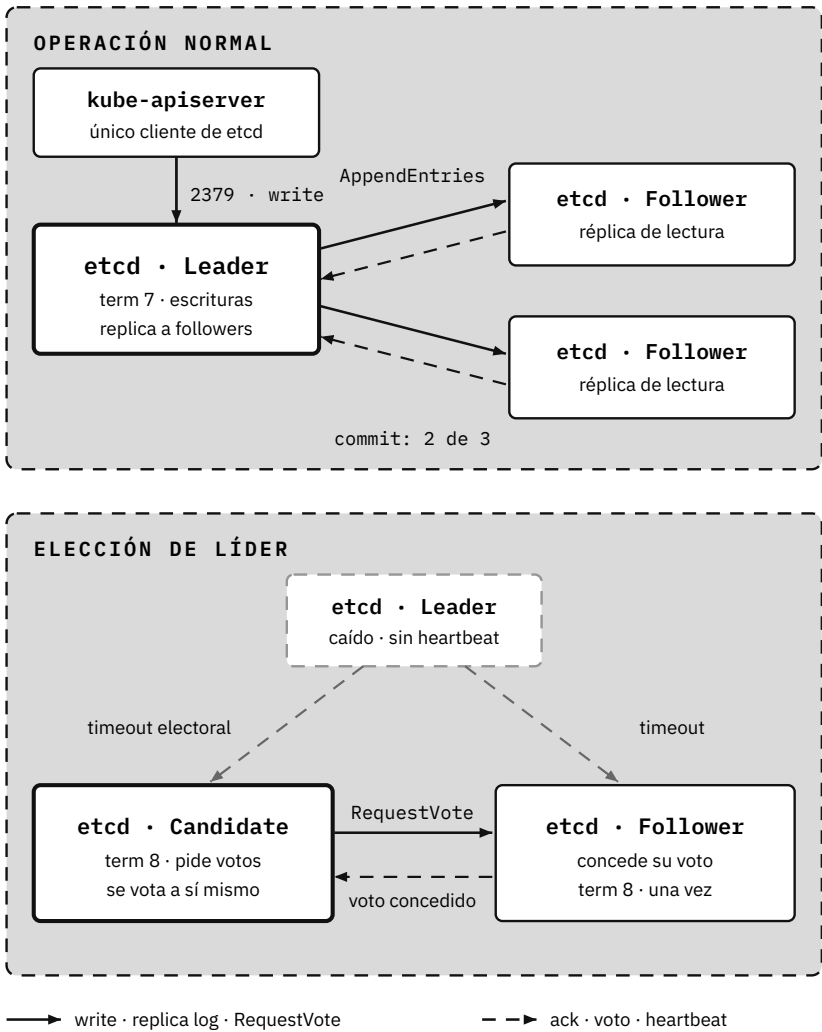


Figura 53: etcd: elección de líder con Raft

Una escritura se confirma cuando la acepta la mayoría, no todos: en un clúster de tres, con dos basta. Por eso etcd se despliega siempre en número impar, y por eso perder dos de tres miembros deja el clúster en solo lectura.

kube-controller-manager

El kube-controller-manager es un único proceso que ejecuta internamente múltiples controladores (también llamados control loops), cada uno responsable de monitorizar un tipo de recurso y de asegurarse de que se encuentra en el estado correcto.

A continuación, algunos de los controladores que corren dentro del kube-controller-manager. Los recursos que gestionan los has ido viendo capítulo a capítulo; lo que enseña esta lista es que no hay un proceso por cada uno: son bucles dentro del mismo binario.

- **Certificate Controller:** gestiona automáticamente la emisión y renovación de certificados.
- **CronJob Controller:** ejecuta jobs según una programación temporal definida.
- **DaemonSet Controller:** asegura que todos los nodos (o algunos filtrados) ejecuten una copia de un Pod específico.
- **Deployment Controller:** gestiona el despliegue declarativo de aplicaciones y sus actualizaciones progresivas.
- **EndpointSlice Controller:** crea y actualiza objetos EndpointSlice que conectan Services con Pods.
- **HorizontalPodAutoscaling Controller:** escala automáticamente el número de Pods basándose en métricas definidas.
- **Job Controller:** supervisa y gestiona tareas de un solo uso hasta su finalización.
- **Namespace Controller:** gestiona el ciclo de vida de los Namespaces y su limpieza.
- **Node Controller:** monitoriza los nodos y responde cuando no están disponibles.
- **ReplicaSet Controller:** mantiene en ejecución el número de réplicas declarado por cada ReplicaSet.
- **ResourceQuota Controller:** aplica las restricciones de recursos por Namespace.
- **Service Account Controller:** crea cuentas de servicio predeterminadas en nuevos Namespaces.
- **StatefulSet Controller:** gestiona el despliegue y escalado de aplicaciones con estado.

cloud-controller-manager

Cuando el clúster se ejecuta sobre un proveedor cloud (AWS, GCP, Azure...), existe un componente adicional llamado `cloud-controller-manager`. Su función es aislar la lógica específica de cada proveedor del resto del control plane, de forma que el `kube-controller-manager` pueda seguir siendo independiente del proveedor sobre el que se ejecuta.

Entre sus responsabilidades habituales se encuentran:

- **Node Controller:** comprueba en el proveedor cloud si un nodo ha sido eliminado tras dejar de responder.
- **Route Controller:** configura las rutas de red necesarias en la infraestructura del proveedor.
- **Service Controller:** crea, actualiza y elimina los load balancers del proveedor cuando se crean Services de tipo LoadBalancer.

Cómo se ejecutan los contenedores: CRI, runtimes y OCI

Cada nodo necesita un container runtime: el software responsable de descargar imágenes, crearlas y ejecutar contenedores. Kubernetes no implementa esta lógica directamente, sino que se comunica con el runtime a través de la Container Runtime Interface (CRI), una API estándar que permite usar cualquier implementación de runtime sin modificar `kubelet`. Los más utilizados actualmente son `containerd` y CRI-O.

No siempre fue así. En sus primeras versiones, Kubernetes integraba Docker de forma totalmente acoplada: el `kubelet` llamaba directamente a la API de Docker, sin ninguna capa de abstracción de por medio. Si querías utilizar otro runtime, no había forma de hacerlo. CRI apareció en la versión 1.5 precisamente para romper ese acoplamiento. Como Docker no hablaba CRI de forma nativa, se añadió `dockershim`, una capa de traducción dentro del propio `kubelet` que permitía seguir usando Docker mientras el ecosistema migraba hacia runtimes que sí la implementaban de forma nativa, como `containerd` o CRI-O. Finalmente, en la versión 1.24 (2022), `dockershim` se eliminó del `kubelet`; Docker dejó de soportarse “de fábrica”, aunque sigue siendo posible usarlo a través de `cri-dockerd`, un adaptador mantenido por la comunidad.

CRI resuelve el problema de cómo se comunican el kubelet y el runtime que ejecuta los contenedores, pero ¿cómo se resuelve el problema de ejecutar contenedores con distintos niveles de aislamiento dentro del mismo nodo? Aquí es donde entra RuntimeClass. En la práctica, un nodo ejecuta una única implementación CRI (por ejemplo, containerd), no varias a la vez. Lo que RuntimeClass permite seleccionar, por Pod, es qué runtime de bajo nivel (handler OCI) debe usar esa implementación CRI para ejecutar los contenedores de dicho Pod: por ejemplo runc (el estándar, basado en namespaces y cgroups de Linux), Kata Containers (aislamiento mediante una VM ligera) o gVisor (un kernel de aplicación en espacio de usuario que intercepta y reimplementa las syscalls). Esto resulta útil cuando se necesita ejecutar cargas de trabajo no confiables con mayor aislamiento, sin tener que dedicarles un nodo o clúster aparte.

Dos retiradas anunciadas que conviene tener en el radar. Esta capa lleva años siendo invisible y se hace notar justo al actualizar. La primera: la retirada de los flags de configuración del kubelet, que estaba prevista para la 1.37, se ha aplazado a la **1.38** precisamente para que coincida con el fin de soporte de containerd v1.7; desde la 1.37, kubeadm ya avisa en su preflight si el runtime instalado se queda corto. La segunda: cgroup v1, deprecado desde la 1.35, sigue funcionando sin cambios en la 1.37, pero su eliminación está planificada (KEP-5573). Las dos se comprueban sobre los nodos, no sobre el control plane, y las dos son de las que conviene mirar antes de subir de versión y no después.

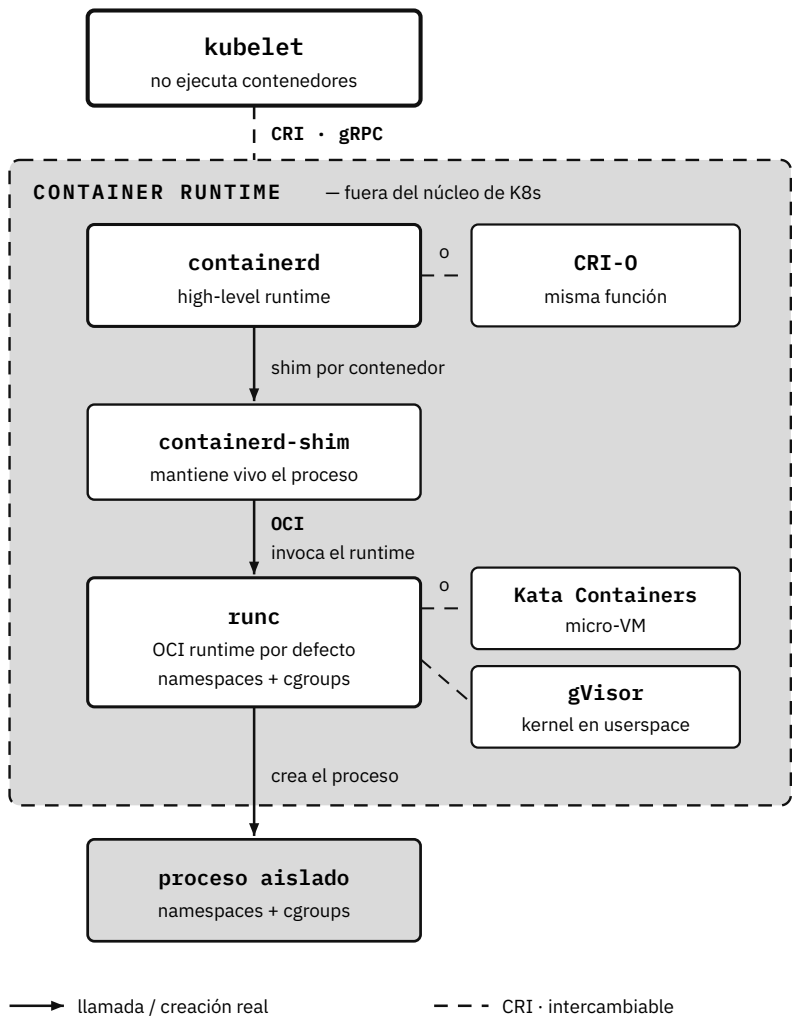


Figura 54: Del kubelet al contenedor: CRI, runtimes y OCI

Las dos interfaces existen para que las piezas sean intercambiables: puedes sustituir containerd por CRI-O, o runc por gVisor o Kata, sin que Kubernetes se entere.

Leases

Ya están todas las piezas sobre la mesa, y con ellas aparecen dos preguntas que ningún componente por separado puede responder: si un nodo lleva un rato callado, ¿está caído o solo lento? Y si el `kube-scheduler` corre en tres réplicas para no tener un punto único de fallo, ¿cuál de las tres manda?

Kubernetes resuelve las dos con el mismo objeto, y es de los más pequeños de la API. Un Lease representa un **testigo con fecha de caducidad**: un componente declara que lo posee durante unos segundos y debe renovarlo periódicamente. Si deja de renovarlo, se asume que ha fallado.

- **¿Quién está vivo?** En un sistema distribuido no se puede distinguir un componente caído de uno lento; la única respuesta práctica es exigir señales periódicas. Los `kubelets` renuevan su Lease (`kube-node-lease`) cada pocos segundos y, si dejan de hacerlo, el nodo pasa a `NotReady`.
- **¿Quién está al mando?** Cuando se ejecutan varias réplicas de un componente para tener alta disponibilidad, solo una debe estar activa. El Lease actúa como un lock distribuido: quien consigue renovarlo es el líder, y si falla, otra réplica toma el relevo.

Por qué importan:

- *Escalabilidad*: antes los `kubelets` reportaban su salud actualizando el objeto `Node` completo, una escritura grande y frecuente contra `etcd`. Sustituirla por un Lease redujo drásticamente ese tráfico y es lo que hace viables los clústeres de miles de nodos.
- *Alta disponibilidad*: sin leader election, dos schedulers programarían el mismo Pod. Los Leases dan ese lock reutilizando `etcd`, sin infraestructura adicional.
- *Simplicidad*: un único primitivo cubre detección de fallos, elección de líder y descubrimiento de instancias.

Ejemplos de uso en el control plane:

- `kube-apiserver` (desde 1.26) publica su presencia mediante Leases.
- `kube-controller-manager` y `kube-scheduler` los usan para leader election (solo uno es activo aunque se ejecuten en HA).
- Los `kubelets` crean y renuevan su Lease en `kube-node-lease` para informar de su estado.

Ese kube-node-lease es uno de los cuatro Namespaces que Kubernetes trae de fábrica, y lo verás enseguida en el capítulo siguiente. Si alguna vez quieres mirar los testigos por dentro: `kubectl get lease -n kube-node-lease`.

Laboratorio. Módulo *El clúster por dentro*, el que cierra la parte práctica: ahí ves en marcha lo que este apéndice cuenta.

HASTA AQUÍ EL REGALO

Quedan trece capítulos

Acabas de leer el arranque de Kubernetes 101: la guía en español para entender Kubernetes desde cero.

- **14 capítulos**

De qué es un contenedor a observabilidad.

- **6 apéndices**

Glosario, el sistema completo, el estado por versión,

el clúster por dentro y una chuleta de kubectl.

- **51 laboratorios**

Un clúster real en tu navegador. Sin instalar nada.

- **5 retos**

Te entregan un clúster roto. Tú lo arreglas.

Disponible el 1 de septiembre

Kindle 12,99 EUR • Papel 24,99 EUR

Reservar en Amazon

www.javivela.dev/libros/kubernetes-101

© 2026 Javier Vela Aylón • Extracto de Kubernetes 101 • Uso personal